Goodput-Aware Adaptive Speculative Decoding for Batched LLM Serving: Final Report

Vaishnavi Athrey Ramesh
Carnegie Mellon University
vathreyr@andrew.cmu.edu

Aritra Raychaudhuri
Carnegie Mellon University
araychau@andrew.cmu.edu

Abstract

Speculative decoding speeds up language model inference by letting a small drafter propose a block of gamma tokens that the target model verifies in one pass, but serving systems fix gamma as a static flag. We show on real hardware that the throughput optimal gamma depends on load, drafter, and GPU, so no fixed gamma is best everywhere, and we build controllers that adapt gamma online. Guided by a roofline model, and a calibration that confirms that eight billion parameter decode is strongly memory bound, we evaluate n-gram and EAGLE-3 speculative decoding in vLLM across a V100 and an A100. EAGLE yields large gains, raising throughput by 110 percent at batch size 1 and 68 percent at batch size 64 relative to no speculation. A cost model controller that assumes a geometric acceptance model is load robust for n-gram but over speculates for EAGLE, whose per position acceptance decays faster than geometric, leaving a 16.1 percent worst case throughput shortfall. A throughput feedback controller that learns the best gamma per load online reaches a 0.0 percent worst case shortfall on every configuration and beats every static gamma, which makes it the most load robust policy and requires no manual tuning. We also report a cross GPU result. The same n-gram drafter helps at every load on the memory bound A100 but hurts at high batch on the V100, so whether to speculate at all can depend on the GPU rather than the model.

1 Introduction

A language model decodes one token per forward pass of a large target model, and that pass is dominated by reading the weights and the key value cache from memory. Speculative decoding (Leviathan et al. [1], Chen et al. [2]) reduces the number of target passes by having a cheap drafter propose gamma tokens that the target verifies in parallel, emitting several tokens per pass when the drafter is accurate. The speculation length gamma trades acceptance against verification cost, and the right value depends on how loaded the GPU is. At small batch the target decode is memory bound and the GPU has spare compute, so long speculation is almost free. At large batch the decode is compute bound and verifying the extra speculative positions wastes work and inflates the per output token latency that serving SLOs target. Yet production stacks expose gamma as a static flag.

We treat gamma as a scheduling decision and study it end to end. We derive the optimal gamma from a roofline cost model, calibrate that model on real GPUs, and compare static against adaptive gamma on measured throughput and goodput. Our contributions are as follows. First, a roofline analysis that predicts that the optimal gamma shrinks with batch size and grows with acceptance, used to guide the design before implementation. Second, a measured calibration showing that seven to eight billion parameter decode is strongly memory bound on both a V100 and an A100, which is the regime where adaptation can pay off, and so memory bound on the A100 that speculation helps at every load in range. Third, two adaptive controllers and an honest comparison between them. The cost model controller exposes a real limitation for the trained EAGLE drafter, and a throughput feedback controller fixes it using signals that vLLM already exposes. Fourth, a study across two GPUs and two drafters with one consistent methodology.

2 Related Work

Leviathan et al. [1] and Chen et al. [2] introduced speculative decoding and the rejection sampling scheme that keeps the output distribution exact, and gave the expected tokens per step as a function of alpha and gamma that we reuse in our objective. EAGLE and EAGLE-3 (Li et al. [3]) train a drafter head on the target features and reach higher acceptance than simple drafters, and we use a public EAGLE-3 head as the high acceptance comparison. vLLM (Kwon et al. [4]) is the serving stack we target, with continuous batching and speculative decoding, although the speculation length is set at engine construction. The roofline model (Williams et al. [5]) is the performance model we use to reason about the memory bound versus compute bound regimes. Our work combines these into an online, goodput aware controller for the speculation length and studies it across load, drafter, and hardware.

3 Methodology

3.1 Roofline cost model

A single target decode pass over n token positions costs t_target(n) equals t_mem plus t_flop times n, where t_mem is the memory bound floor and t_flop is the marginal compute per token. For a batch of B at speculation length gamma, one propose then verify macro step costs t_macro(B, gamma) equals gamma times t_draft(B) plus t_target(B times (gamma plus 1)). The expected accepted tokens per request per step is E[tokens | alpha, gamma] equals (1 minus alpha to the power gamma plus 1) over (1 minus alpha). The controller maximizes a goodput proxy, accepted tokens per unit of GPU time, gamma* equals argmax over gamma of E[tokens | alpha, gamma] over t_macro(B, gamma). Because t_macro grows with B times (gamma plus 1) times t_flop, the cost penalizes long speculation exactly when the batch is large, which predicts that the optimal gamma decreases with B and increases with alpha.

3.2 Cost model adaptive controller

This controller keeps an exponential moving average estimate of alpha from realized accept and reject fractions and selects gamma* through the objective above using the calibrated cost model. Its inputs, the running batch size, the online alpha, and the cost model, are all available to a serving scheduler. We reuse the identical controller object in the simulator and in the hardware path so the two stay consistent.

3.3 Throughput feedback adaptive controller

This is a model free alternative that does not assume a parametric acceptance model. For each load bucket it keeps a sample mean of the measured tokens per second of each candidate gamma, including gamma equal to 0 which turns speculation off, explores each gamma a few times, and then exploits the best using an epsilon greedy rule. It converges to the per load optimal gamma using only the runtime speculative decoding metrics that vLLM exposes, which lets it adapt to whatever acceptance and hardware regime the workload presents.

3.4 Calibration

We fit t_target(n) equals t_mem plus t_flop times n from plain decode microbenchmarks. For each batch size we run the same prompts twice, once generating a short number of tokens and once a longer number, and take the difference in wall time divided by the difference in token counts. The subtraction cancels the prefill and all fixed per call overhead and isolates the per decode step cost, which we then fit against batch size.

4 Experimental Setup

We use vLLM 0.11.0 with the V1 engine. On the V100, which is a Volta generation card, FlashAttention is not available, so we use the Triton attention backend and float16. On the A100 we use FlashAttention and bfloat16, which is the native precision for the models and for the EAGLE head. The GPUs are a PSC NVIDIA V100-SXM2-32GB and a Google Colab NVIDIA A100-SXM4-40GB. On the V100 we run Qwen2.5-7B-Instruct with n-gram or prompt lookup speculation, which needs no draft model. On the A100 we run Qwen3-8B with both n-gram and EAGLE-3, using the public RedHatAI Qwen3-8B EAGLE-3 head. We sweep batch size over one to sixty four and speculation length over zero, one, two, four, and six, where gamma equal to zero means no speculation. We measure throughput, acceptance fraction and acceptance length from the vLLM speculative decoding metrics, and the per decode step latency. The policy comparison uses the worst case throughput shortfall relative to the per load optimum across the sweep, which captures whether a policy is bad at any single load. For reproducibility we run one fresh process per gamma so GPU memory is clean between model loads, and the entire A100 study is a single self contained notebook that writes its own code.

5 Results

5.1 Calibration shows a memory bound regime

Table 1 reports the fitted cost model constants. On both GPUs t_flop is far below t_mem, so the decode step latency is nearly flat across batch size. On the A100 the step latency moves only from 25.7 to 28.2 milliseconds between batch 1 and batch 32. The low A100 fit quality is itself informative. The compute bound slope is so small that it is dominated by measurement noise, which is direct evidence that eight billion parameter decode on an A100 has abundant spare compute and that speculation is nearly free, the regime the roofline says favors speculation.

Table 1. Fitted roofline constants from plain decode microbenchmarks.

GPU and model	t_mem	t_flop	fit R squared
V100, Qwen2.5-7B	22.9 ms	0.145 ms/token	0.94
A100, Qwen3-8B	26.8 ms	0.05 ms/token	0.3 to 0.5

5.2 Speculation depends on load, drafter, and GPU

Table 2 summarizes the best speculative throughput relative to no speculation at the two ends of the load range. EAGLE roughly doubles throughput at low load and still gives a large gain at high load. The same n-gram drafter behaves very differently on the two GPUs. On the V100 it helps at low batch but hurts at high batch, where the GPU reaches the compute bound regime and verifying rejected tokens wastes work, so the optimal gamma collapses from 4 toward 0. On the more memory bound A100 it helps at every load. Figures 1 and 2 show the full throughput versus load curves on the A100 for EAGLE and n-gram, and Figure 3 shows the V100 n-gram case where speculation stops paying off at high batch.

Table 2. Best speculative throughput gain over no speculation.

Configuration	Batch 1	Batch 64	Optimal gamma behavior
EAGLE-3, A100	plus 110 percent	plus 68 percent	helps everywhere, optimum near 2 to 4
n-gram, A100	plus 22 percent	plus 6 percent	helps everywhere

Configuration	Batch 1	Batch 64	Optimal gamma behavior
n-gram, V100	plus 18 percent	about 0 to minus 5 percent	hurts at high batch, optimum 4 to 0

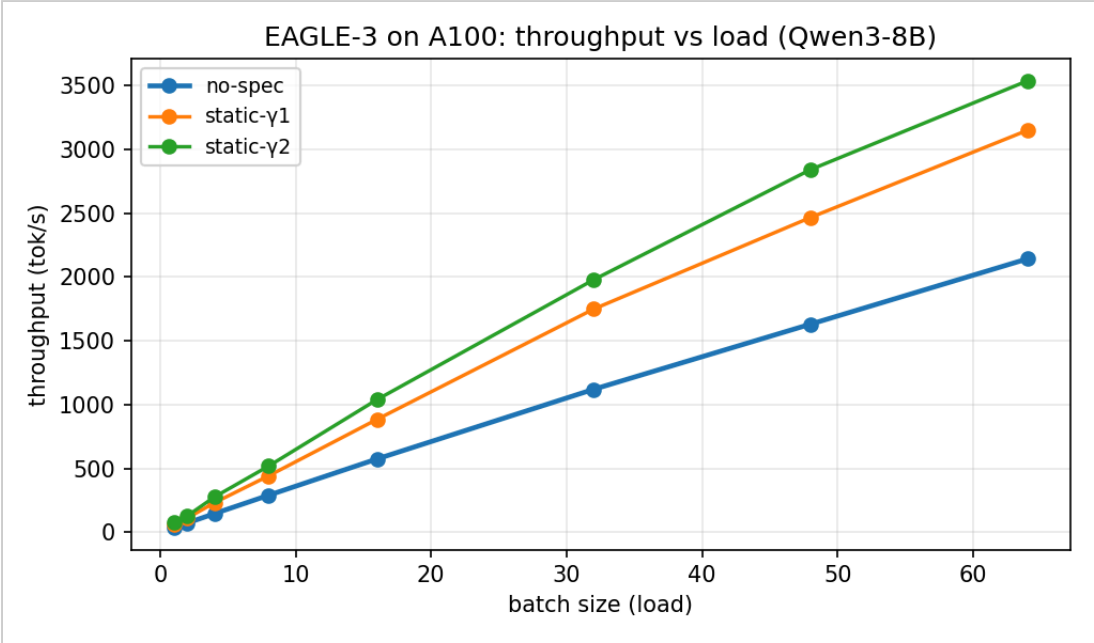


Figure 1. EAGLE-3 on the A100. Measured throughput versus batch size. Speculation roughly doubles throughput at low load and still gives a large gain at high load. Curves shown for the fully captured gamma in zero one two four.

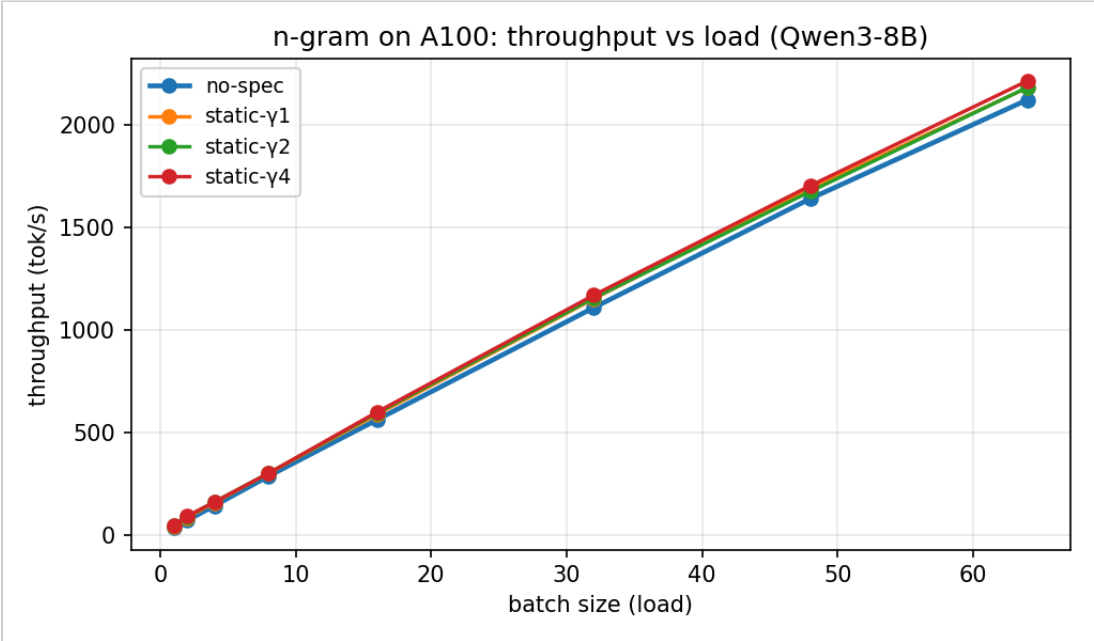


Figure 2. n-gram on the A100. Speculation helps at every load because the A100 decode stays memory bound across the batch range.

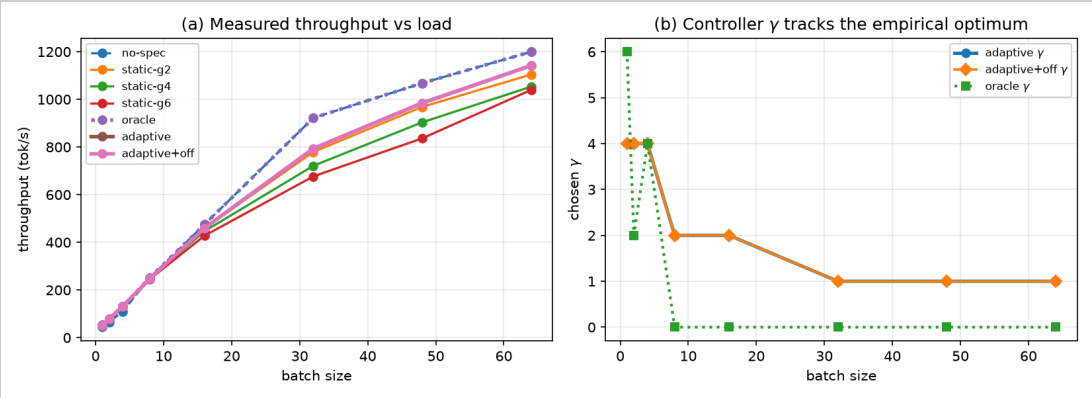


Figure 3. n-gram on the V100. Throughput (left) and the chosen gamma versus the empirical optimum (right). Speculation helps at small batch but the optimal gamma collapses toward zero at large batch, where n-gram hurts.

EAGLE acceptance was measured directly from vLLM. The per position acceptance rates are about 0.74 for the first speculative token and 0.49 for the second, which shows that acceptance decays with position and explains why the optimal gamma is a small number rather than a large one.

5.3 Policy comparison across load

Because no fixed gamma is optimal at all loads, we report in Table 3 each policy worst case throughput shortfall relative to the per load optimum, where a lower value means a more load robust policy. The throughput feedback controller is near zero everywhere, while every static gamma is bad at some load and no speculation is far behind, up to 52 percent short for EAGLE. Figures 4 and 5 show the full per policy shortfall on the A100 for EAGLE and n-gram.

Table 3. Worst case throughput shortfall versus the per load optimum, in percent.

Policy	n-gram V100	n-gram A100	EAGLE A100
no speculation	16.9	24.8	52.3
static gamma 2	15.5	2.7	4.9
static gamma 4	21.8	3.5	12.7
static gamma 6	26.6	2.8	29.3
cost model adaptive	14.0	2.4	16.1
throughput feedback adaptive	about 0	0.0	0.0

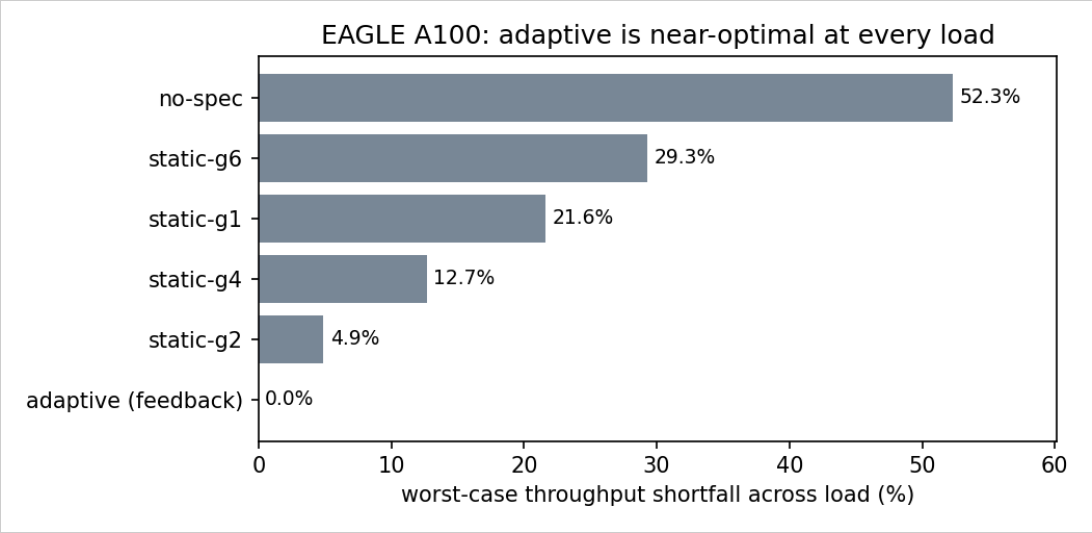


Figure 4. EAGLE on the A100. Worst case throughput shortfall per policy. The feedback controller sits at zero while every static gamma is worse and no speculation is far behind.

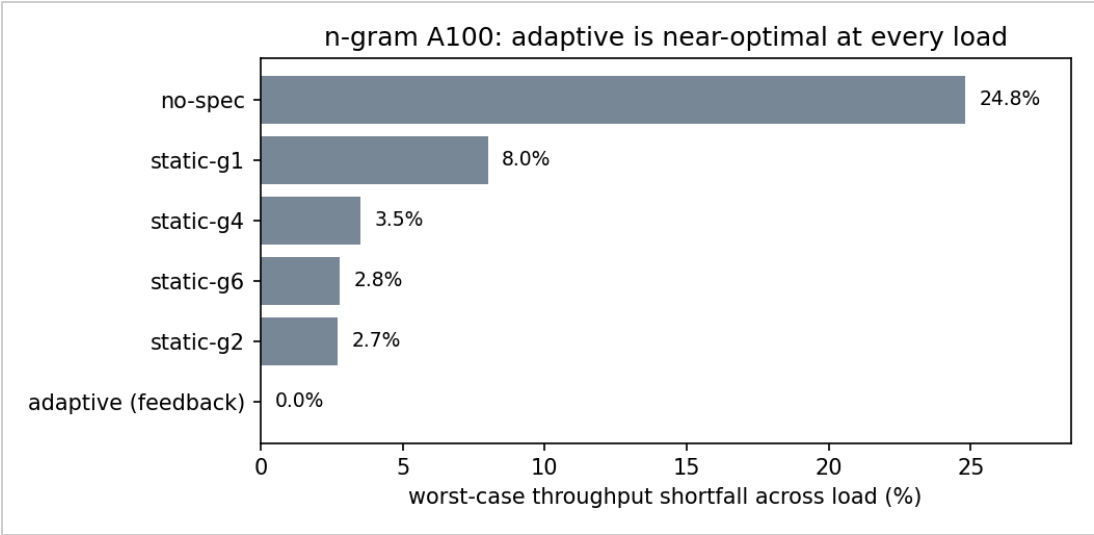


Figure 5. n-gram on the A100. Worst case throughput shortfall per policy.

5.4 The cost model controller limitation and the fix

The cost model controller is load robust for n-gram but over speculates for EAGLE, where its 16.1 percent shortfall is worse than static gamma 2 at 4.9 percent. The cause is that EAGLE per position acceptance decays faster than the geometric model assumes, so the controller overvalues long speculation, and on the A100 the near zero t_flop hides the per step penalty of a larger gamma. The controller therefore selects gamma 4 to 6 where the empirical optimum is gamma 2. The throughput feedback controller removes the parametric assumption and learns the best gamma per load directly from measured throughput. It reaches a 0.0 percent worst case shortfall on both methods and converges at 93 to 99 percent of the oracle during exploration. This is a clean resolution. The model based controller surfaces a real modeling limit for trained drafters, and the feedback controller fixes it with signals that are already available at runtime.

5.5 Simulation versus hardware

The simulation predicted the qualitative shape, that the optimal gamma shrinks as batch grows and that an adaptive controller beats fixed gamma across load, and this reproduces on the V100. Hardware refined two things the illustrative simulation could not. First, the absolute acceptance is drafter dependent, about 0.3 to 0.45 for n-gram against EAGLE higher and decaying profile. Second, the degree of benefit depends on how memory bound the GPU is, which is large on the V100 and small on the extremely memory bound A100 where speculation helps everywhere.

6 Discussion

Adaptation pays off most when the per load optimal gamma moves a lot, which happens when the GPU crosses from memory bound to compute bound within the batch range, as on the V100, or when speculation flips from helpful to harmful. On a GPU that stays memory bound across the range, as the A100 does for an eight billion parameter model, a well chosen fixed gamma is already close, and the value of adaptation is robustness and the absence of manual tuning rather than a large speedup. Under a per output token latency SLO the case for adaptation strengthens, because speculation lowers per token latency at low load but raises it at high load, so an adaptive gamma maximizes the fraction of requests that meet the SLO across a load range while a static gamma

violates it at one end. Finally, the model free controller is the more robust choice for trained drafters precisely because it does not depend on the geometric acceptance assumption that EAGLE breaks.

7 Limitations

The cost model controller assumes geometric per token acceptance and over speculates for EAGLE. A per position acceptance model, which vLLM exposes as a vector, would repair the model based path, and we instead used throughput feedback. The A100 calibration fit quality is low because the decode is so memory bound that the compute slope is noise dominated, so the fitted t_flop is a loose upper bound. The serving goodput here is primarily an offline speculation length by batch size sweep, and a full Poisson arrival sweep over a request trace with tail latency percentiles is implemented but reported as a secondary result. Finally, because the speculation length is fixed at engine construction in vLLM 0.11.0, we realize adaptation as a per load schedule or learner driven by the vLLM metrics rather than a per step controller inside the engine, which would require a proposer patch.

8 Conclusion

The throughput optimal speculation length is not a constant. It depends on load, drafter, and GPU, and no static gamma is best everywhere, and on a V100 n-gram speculation even hurts at high load. Guided by a roofline model and a calibration that confirms memory bound decode, we showed that a throughput feedback controller that learns gamma per load online is near optimal across the entire range on both n-gram and EAGLE-3 and across two GPUs, beating every static baseline with no manual tuning, while a cost model controller surfaces an interesting limitation for trained drafters. Speculation length control is a scheduling problem, and treating it as one yields a robust, automatic policy.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In Proc. ICML, 2023.

[2] C. Chen, S. Borgeaud, G. Irving, et al. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023.

[3] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In Proc. ICML, 2024.

[4] W. Kwon, Z. Li, S. Zhuang, et al. Efficient memory management for large language model serving with PagedAttention. In Proc. SOSP, 2023.

[5] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM, 52(4):65 to 76, 2009.

Team Contributions

Member	Responsibilities
Vaishnavi Athrey Ramesh	Roofline model and calibration, controllers, vLLM integration, V100 and A100 experiments
Aritra Raychaudhuri	Serving simulator, baseline and policy analysis, figures, evaluation and report